`auto` Type Inference in a Single-Pass Compiler

Adding initialiser-driven inference to Selfie without an AST

Florian Schroffner
Department of Computer Sciences, University of Salzburg
Advanced Systems Engineering
Supervisor: Prof. Christoph Kirsch

June 24, 2026

Abstract

This paper extends the Selfie/C* compiler with a single keyword, `auto`, for initialiser-driven local type inference — added *without* changing Selfie's single-pass architecture. The central observation is that Selfie's single-pass parser already performs lightweight type tracking: every expression-compiling function returns the type (`uint64_t` or `uint64_t*`) of the value it just produced. The `auto` feature simply captures the type this existing machinery computes for the initialiser and records it in the symbol table. No abstract syntax tree, no separate type-checking pass, and no change to the code generator are required. The implementation touches three functions in `selfie.c`. The extended compiler still self-compiles to a 43 728-instruction RISC-U binary and passes the project autograder, including a dedicated `auto`-type-inference suite with positive (compile-and-run) and negative (must-reject) tests. The project demonstrates that a feature usually associated with multi-pass compilers does not, in its initialiser-driven form, require multi-pass machinery.

1 Introduction

Type inference — the ability of a compiler to deduce the type of a variable from its initialiser — is a feature taken for granted in modern languages. C++ offers `auto`, Go provides `:=`, and Rust infers types across entire functions. A common assumption is that adding such inference to a compiler requires a multi-pass pipeline with an explicit Abstract Syntax Tree (AST) and a dedicated semantic phase that can reason about types before code generation.

This paper shows that the assumption is unnecessary for *initialiser-driven* inference. Selfie/C* [1], the basis of this work, interleaves parsing and code emission in a single pass and has only two types (`uint64_t` and `uint64_t*`). Crucially, it already performs lightweight type tracking: every expression-compiling function *returns* the type of the value it has just compiled. The type needed to give an `auto` variable a concrete type is therefore already computed at the moment its initialiser is parsed.

This work extends Selfie with the `auto` keyword for local variables. It answers three research questions:

RQ1. Can initialiser-driven `auto` inference be added to a single-pass compiler without introducing an AST or a separate semantic pass?

RQ2. What is the smallest set of changes to `selfie.c` that implements `auto` correctly, including stack allocation and procedure cleanup?

RQ3. Which inference patterns does the feature cover, and how does it compare to `auto`/`var`/`:=` in mainstream languages?

The remainder of this paper is structured as follows. Section 2 reviews Selfie/C*. Section 3 specifies the `auto` extension. Section 4 shows how the single pass already supplies the needed type information. Section 5 details the three-function implementation. Section 6 evaluates the result through self-compilation and the autograder. Section 7 concludes.

2 Background

2.1 Selfie and C*

Selfie [1] is a self-referential system developed at the University of Salzburg. It comprises a compiler for the C* language, a RISC-U emulator (mipster), and a hypervisor (hypster), all written in approximately 10 000 lines of C. C* is a strict subset of C with only two types, no floating-point arithmetic, and no separate compilation.

The Selfie compiler is a single-pass recursive-descent parser: as tokens are consumed, RISC-U machine code is emitted immediately. There is no explicit AST, so there is no obvious place for a *whole-program* semantic phase. It does, however, maintain enough type information locally to type-check assignments and arithmetic: each expression-compiling function returns a type tag (`UINT64_T` or `UINT64STAR_T`) describing the value it just emitted. This local, already-present type tracking is exactly what `auto` reuses.

2.2 RISC-U

RISC-U is a minimal subset of the RISC-V ISA defined for Selfie. It consists of 14 instructions: `lui`, `addi`, `add`, `sub`, `mul`, `divu`, `remu`, `sltu`, `ld`, `sd`, `beq`, `jal`, `jalr`, and `ecall`. Its simplicity makes it an ideal compilation target for educational compilers: the full ISA fits on one page, yet it is sufficient for Turing-complete programs.

3 The `auto` Extension

The `auto` extension keeps C* a strict subset: every valid C* program still compiles unchanged. The only addition is the `auto` keyword for local variables whose type is inferred from the initialiser. No new type keywords are introduced.

3.1 The Two Inferable Types

`auto` infers one of Selfie's two existing types. It adds no types, widths, or conversions; the inferred type is precisely what Selfie's expression compiler already produces for the initialiser.

Table 1: The two types `auto` can infer

Type	Internal tag	Inference Example	Inferred Type
`uint64_t`	`UINT64_T`	`auto y = 1;`	`uint64_t`
`uint64_t*`	`UINT64STAR_T`	`auto p = malloc(8);`	`uint64_t*`

Because there is no second numeric type, there is no promotion rule and no implicit conversion to specify: the type of an `auto` variable is identical to the type Selfie would assign to the right-hand side of an ordinary assignment.

3.2 Pointer Rules

The pointer rules are inherited verbatim from C*; `auto` merely reports their result.

Table 2: Pointer type rules (unchanged from Selfie)

Expression	Condition	Result Type
`auto v = *p`	`p : uint64_t*`	`uint64_t`
`auto q = p + n`	`p : uint64_t*, n : uint64_t`	`uint64_t*`
`auto d = p - q`	`p, q : uint64_t*`	`uint64_t`
`auto r = malloc(n)`	`n : uint64_t`	`uint64_t*`
`p + q`	`p, q : uint64_t*`	**Error**

3.3 Example Program

`auto` declarations are ordinary statements inside any procedure body, including `main`.

Listing 1: A complete program using `auto`

```
uint64_t factorial(uint64_t n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

uint64_t main() {
    auto n   = 5;              // uint64_t
    auto res = factorial(n);   // uint64_t  -- callee return type
    auto p   = malloc(8);      // uint64_t* -- malloc result
    *p = res;
    auto v   = *p;             // uint64_t  -- dereference
    return v - 120;            // 0 on success
}
```

3.4 Restrictions From the Single Pass

The properties that keep inference simple are not enforced by extra checks; they follow from the single-pass design.

(a) Direct initialiser only.
The type of an `auto` variable is exactly the type of its own initialiser. Types never flow back from later uses, so no constraint solving is ever required.

(b) No type hierarchy.
Inference yields one of two types; there is no promotion lattice to search, and mixed-type arithmetic is governed by Selfie's pre-existing rules.

(c) No circular or forward `auto`.
In a single pass an identifier must be declared before use, so a variable cannot appear in its own or a not-yet-declared variable's initialiser. Such cases are rejected automatically as undeclared-identifier errors.

4 Compiler Architecture

Selfie's single-pass design — scanner, parser, and code emitter advancing in lock-step — is left unchanged. The pipeline is:

`Scanner ⟶ Recursive-descent parser + code emitter ⟶ RISC-U`

There is no AST; the "tree" exists only implicitly in the call stack of the recursive-descent parser. The enabling observation is that Selfie's expression-compiling functions already *return a type*. As `compile_expression()` (and the precedence levels below it, `compile_arithmetic()`, `compile_term()`, `compile_factor()`) emit code that computes a value into a temporary register, each returns a `uint64_t` type tag describing that value. A dereference turns `UINT64STAR_T` into `UINT64_T`; `malloc` yields `UINT64STAR_T`; a string literal yields `UINT64STAR_T`; a call yields the callee's declared return type; pointer arithmetic preserves `UINT64STAR_T`.

By the time the parser finishes compiling an initialiser, the concrete type of that expression is sitting in the return value of `compile_expression()` — precisely the information an `auto` declaration needs. The `auto` extension does not compute types; it reads back the type Selfie already produced. This is why no AST and no separate pass are required.

5 Implementation

The feature touches three functions in `selfie.c`, plus the one-keyword scanner addition (`SYM_AUTO`).

5.1 Parsing an `auto` Declaration

The new function `compile_auto_declaration()` parses `auto identifier = expression ;`, takes the inferred type from the return value of `compile_expression()`, allocates a stack slot, and records the variable in the local symbol table.

Listing 2: The core of the feature

```
void compile_auto_declaration() {
  char* var_name; uint64_t inferred_type;
  get_symbol();                          // consume 'auto'
  var_name = identifier;
  get_symbol();                          // consume identifier
  get_expected_symbol(SYM_ASSIGN);       // consume '='

  inferred_type = compile_expression(); // type Selfie already returns

  emit_addi(REG_SP, REG_SP, -WORDSIZE); // lazy stack allocation
  emit_store(REG_SP, 0, current_temporary());
  tfree(1);

  number_of_auto_variable_bytes = number_of_auto_variable_bytes +
      WORDSIZE;
  create_symbol_table_entry(LOCAL_TABLE, var_name, line_number,
    VARIABLE, inferred_type, 0, -number_of_auto_variable_bytes);

  get_expected_symbol(SYM_SEMICOLON);
}
```

Allocation is *lazy*: because the number of `auto` variables is not known in advance in a single pass, each declaration pushes its own word onto the stack as it is compiled, rather than reserving space in the prologue.

5.2 Statement Dispatch and Frame Size

`compile_statement()` gains one branch so that an `auto` declaration is accepted wherever a statement is allowed. The only change outside the new function is in `compile_procedure()`: the running

counter `number_of_auto_variable_bytes` is initialised to the declared-locals size, and the combined frame size is passed to the epilogue. Because Selfie's epilogue restores `SP` directly from the frame pointer whenever the frame is non-empty, the exact number of pushed `auto` words never has to be tracked for correctness.

5.3 Error Handling for Free

Because inference reuses Selfie's expression compiler, most error cases are rejected by Selfie's existing diagnostics, with no new code; only the void initialiser needs a single explicit check:

- `auto x = undefined;` — the identifier is in no symbol table, so Selfie reports an undeclared identifier.
- `auto x = y;` where `y` is declared later — in a single pass `y` is not yet visible; this is what makes forward and circular `auto` impossible.
- `auto e = p + q;` where both are pointers — Selfie already rejects pointer + pointer.
- `auto x = f();` where `f` returns `void` — there is no value to bind, so `auto` rejects the void initialiser explicitly.

6 Evaluation

6.1 Self-Compilation

The strongest correctness criterion for a self-hosting compiler is that it compile its own source. Because `selfie.c` is plain C* (it does not use `auto`), the extended compiler still self-compiles unchanged: `./selfie -c selfie.c -o selfie.m` produces a RISC-U binary of **43 728** 64-bit instructions and **14 240** bytes of data (a 189 152-byte loaded image), which executes correctly under mipster. Adding `auto` grows the compiler by only the handful of instructions needed for the new function and statement branch.

6.2 Autograder Results

Two autograder targets are relevant. The `selfie-compile` target verifies that self-compilation still succeeds (all three checks pass — `make selfie`, warning-free `./selfie -c selfie.c`, and `.m` binary creation). The dedicated `auto-type-inference` target exercises the feature directly:

Category	Programs	Result
Positive (compile + run, exit 0)	`auto-fint`, `auto-fint-pointer`, `auto-fstring`, `auto-deref`, `auto-comparison`, `auto-nested`, `auto-function-return`, `auto-pointer-arith`, …	✓ all pass
Negative (must reject)	`invalid-undefined-var`, `invalid-circular-dep`, `invalid-fstring-add`	✓ all rejected

Each positive program both compiles *and* returns exit code 0, so an incorrect inference would change the result; the suite therefore validates the inferred type, not merely acceptance of the syntax.

6.3 Comparison with Selfie

Property	**Selfie/C***	**+ `auto`**
Architecture	Single-pass	Single-pass (unchanged)
Explicit AST	No	No
Separate semantic pass	No	No
Type system	2 types	2 types
`auto` inference	No	Yes
Functions changed	—	3
Self-compilation	Yes	Yes
Target ISA	RISC-U	RISC-U

7 Conclusion

This work demonstrates that a feature usually associated with multi-pass compilers — type inference — does not, in its initialiser-driven form, require multi-pass machinery. The type `auto` needs is a by-product of code generation that Selfie's single pass already computes and discards; capturing it in three functions is enough, with no AST, no separate semantic pass, and no change to the code generator.

The key insight is that decidable, initialiser-driven inference is *already implied* by a compiler whose expression functions return types. The restrictions on the feature — direct initialiser only, two types, no forward references — are not weaknesses but the precise boundary of what a single pass makes available, and the natural error cases fall out of Selfie's existing diagnostics for free.

Future work could give `auto` more to infer by adding a type to Selfie, provide a targeted diagnostic for circular dependencies, or extend `auto` to return types and global declarations — each a small, local extension rather than an architectural change.

References

[1] Kirsch, C. M. et al. *Selfie: Reflections on Self-referential Computer Systems*. Onward! 2017, ACM, 2017.

[2] Milner, R. *A Theory of Type Polymorphism in Programming*. Journal of Computer and System Sciences, 17(3):348–375, 1978.

[3] Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson, 2006.

[4] Waterman, A., Asanović, K. (eds.). *The RISC-V Instruction Set Manual, Volume I: User-Level ISA*. RISC-V Foundation, 2019.